

Final
start

5-4-26

while (true) {
Entry ① while (lock == true); → Test
② LOCK = true; → set
// critical section
Exit LOCK = false
// Remainder section
}

** Test & Set method → hardware → Global variable
lock = false
lock

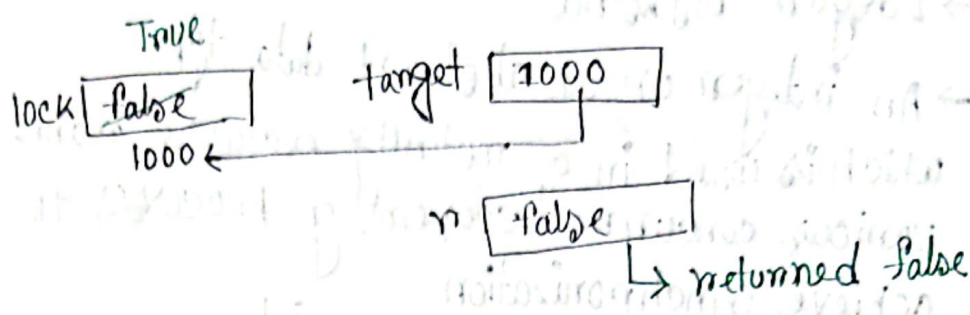
false
1000

while (true) {
Entry while (test-and-set (&lock)); → lock address
// critical section
Exit LOCK = false;
// Remainder section
}

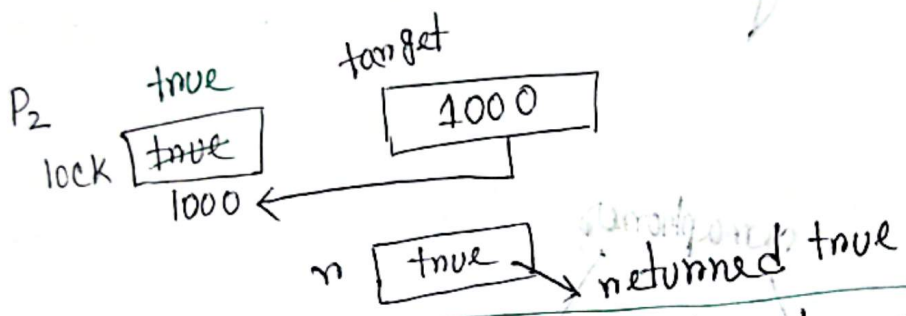
boolean test-and-set (boolean *target) {
boolean r = *target;
*target = true;
return r;
}

atomic
no preemption
not happened
here

P₁



False will be the test on Ending code



Ending code will be true and spinlock happened

read
modify
write → use to make atomic

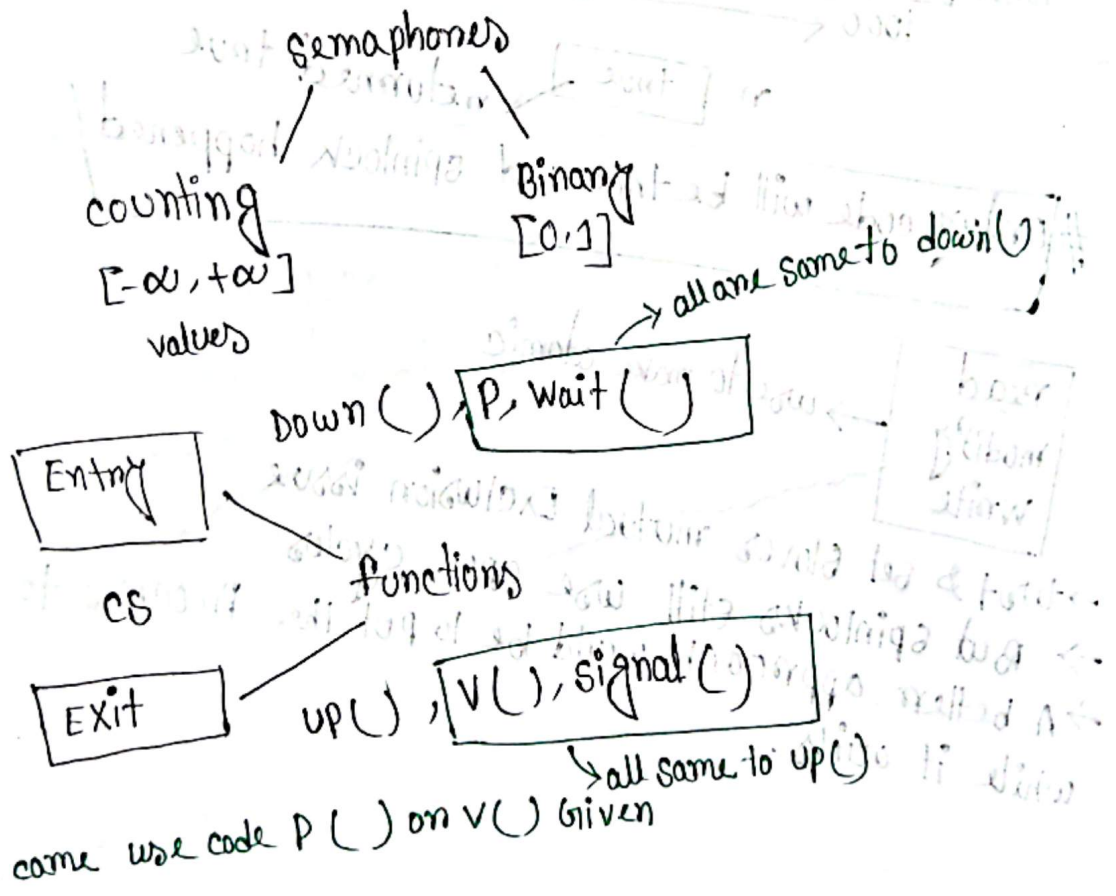
- two & set solves mutual exclusion issue
- But spinlocks still use CPU cycles
- A better approach would be to put the process to sleep() while it waits:

Semaphores (handwritten OS)

→ Edsger Dijkstra

→ An integer on an abstract data type which is used in a mutually exclusive manner by various concurrent/cooperating processes to achieve synchronization

→ A semaphore is just a variable that controls how many processes enter a section of code.



Down (semaphore S) {

S.value = S.value - 1;

if (S.value < 0) {

/* put process in suspend list & sleep() */

{ else {

return;

}

After Down code completion went to the CS

Up (semaphore S) {

S.value = S.value + 1;

if (S.value <= 0) {

/* select a process from suspend list & wake-up() */

{ else {

return;

Down()

{ P1, P2, P3, P4, P5

S [2] 0 -1 -2

CS [P1, P3]

SL [P4, P5]

→ S [3] what mean?

* initial value of S is the max no of processes that can enter CS

* S[0] → this is the last value after this no process can enter in the CS. After 0 all process will be go in a suspend list.

up()

S [2] 1 2 3

CS [P1, P2, P3] P4 P5

SL [P4, P5]

S [2]

* How many processes are currently in my suspended list

S[10] → here are 10 processes in my suspended list

S = 01

if S = 10 6P, 4V S = ?

$$10 - 6 = 4$$

$$4 + 4 = 8$$

Binary (Semaphore op) {

Down (Semaphore S) {

if (S.value == 1) {

S.value = 0;

} else {

/* Put Process in suspend list & sleep() */

} return;

Up (Semaphore S) {

if (suspend list is empty) {

S.value = 1;

} else {

/* Select a Process from suspend list & wake up() */

} return;

S = 01 → successful op.
S = 00 → unsuccessful block
if/when happened in binary

S = 10

PI
CS
RA
SL

S = 01
S = 11

RQ ~~P1~~ ~~P2~~ ~~P3~~

S A O

CS Pa

SL $P_2 P_3$

Pg

S ☒ X O

CS $\boxed{P_1}$ P_2 P_3 P_4

SL P/P

When $S=1$ we conditionally push any process in CS.

R8 P4

8 0

$$CS \sqcup PK : P_2$$

SL $\boxed{P_2 P_3}$ P_4

#S=1 only when suspend list is empty

between P_4 and SL always give priority SL first they go through the down U & CS.
 P_4 put last in the SL

→ semaphore counting

→ Semaphore counting
* $S[0]$ value of semaphore is 0 how many processes are in the suspended list? → 0 because when value becomes minus then it gets suspended. But value of S is 0 means none, there is nothing in suspended list. That time we don't need to run code up. We can directly make value of S from 0 to 1.
Processes can enter into the CS successfully? →

→ Q10. How many processes can enter into the CS successfully? → 10

S 10 9876543210-1

Successful Process mean process successfully ^{→ down} enter into the CS, block unsuccessful

$\delta = \boxed{0} - 1 \rightarrow \text{down} \rightarrow \text{suspend}$

final
2nd class

S [17] 5P, 3V, 1P
17-5+3-1=14

7-04-26

8) Each process $P_i \{i=0 \text{ to } 8\}$ executes the following

code repeat
P(mutex)
/* critical section */
V(mutex)
forever

mutex [X] 0 1

CS [P₁ P₀ P₂]

CS [P₃ P₀ P₂]

Process P₉ is abnormal. It executes the following code:

repeat
V(mutex) ^{semaphore variable}
/* critical section */
V(mutex) / P(mutex)
forever

P₉ up reason it's have VIP access in any process it can enter also exit

Initial value of mutex = 1

what is the max number of processes in the critical section at any point of time? → 3 process

case 1: mutex [X] 0 1

CS [P₀]

case 2: P₀ P₉ P₁ P

CS [P₀ P₉ P₁]

mutex [X] 0 1 0

Case 3:

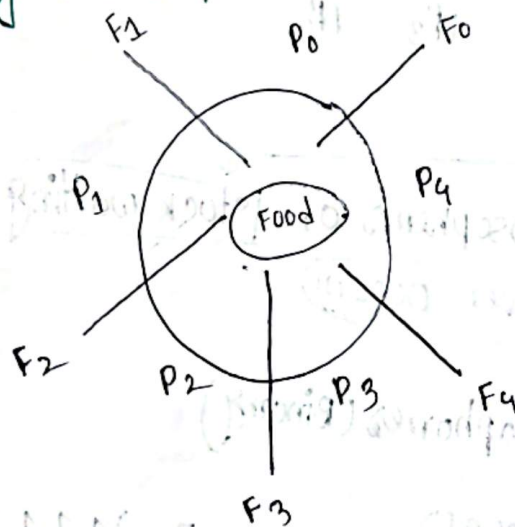
CS $\boxed{P_0 P_1 P_2 P_3} P_4$

mutex $\emptyset \emptyset \emptyset \emptyset \emptyset$

this case to have in their

Ans: 10

Dining Philosophers



- 5 philosophers ($P_0 - P_4$)

- 5 forks ($F_0 - F_4$)

- Every philosopher alternates between thinking and eating forever.

- To eat, every philosopher need 2 forks. They first pick up their left then right fork

Every P_i

while (true) {

thinking();

take-fork(i);

take-fork($(i+1) \% 5$);

eating();

put-fork(i);

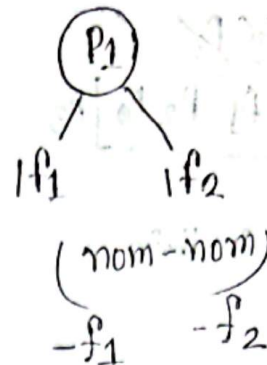
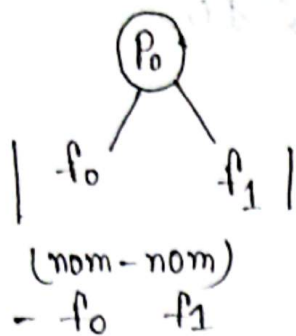
put-fork($(i+1) \% 5$);

}

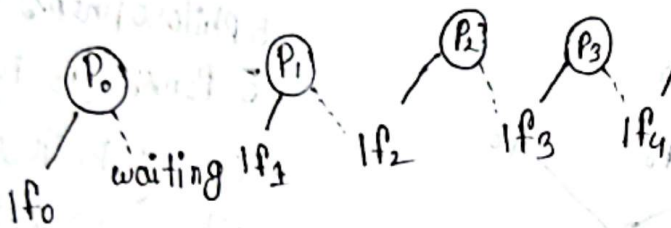
← left fork

→ right fork

Case 1:



Case 2



Dead lock

* why it's Deadlock? (Philosophers are stuck waiting for an event that will never occur)

Model this for Bin Semaphores (Binary)

For 5 forks use 5 semaphores

$int S[5] = \{1, 1, 1, 1, 1\}$

$S_0 - S_4$
(initialized to 1)

S_0	S_1	S_2	S_3	S_4
1	1	1	1	1

$P_0 : S_0 S_1$

$P_1 : S_1 S_2$

$P_2 : S_2 S_3$

$P_3 : S_3 S_4$

$P_4 : S_4 S_0$

for case P_0-P_3

for the case of P_4

```

while (true) {
    thinking ();
    down / P ← wait (Si);
    wait (S(i+1) % 5);
    eating ();
    up / V ← signal (Si);
    signal (S(i+1) % 5);
}

```

```

wait (S(i+1) % 5);
wait (Si);

```

```

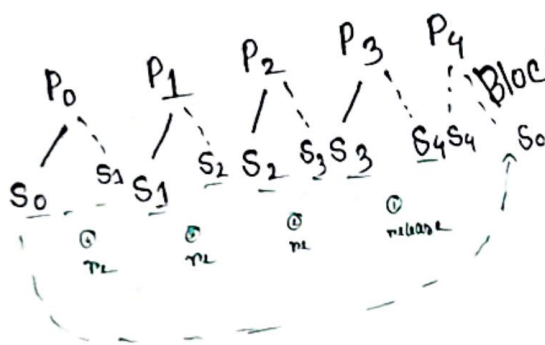
signal (S(i+1) % 5);
signal (Si);

```

Sample solution

For philosophers

P_0-P_3 , left then right fork
 P_4 , right then left fork



Blocked → need S_0 , but already taken by P_0

when P_4 block P_3 get S_4 and then
 S_3 S_4 when release then turn
 0 0
 S_3 S_4 same way going on.
 1 1

Deadlock happen on share resource time

Deadlock

- if 2 or more processes are waiting for some event to occur, which won't even happen, they would be waiting indefinitely (forever). We say that they are stuck in a deadlock.

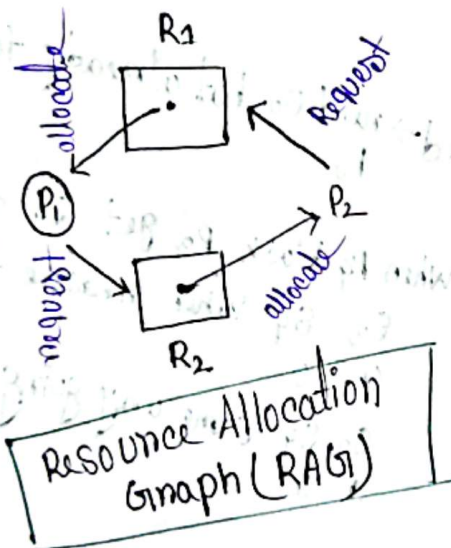
- It's a synchronization problem

P_1 P_2
take up R_1 takes up R_2
sleep() sleep
waiting for R_2 waiting for R_1

Resources

R_1 ☐

R_2 ☐



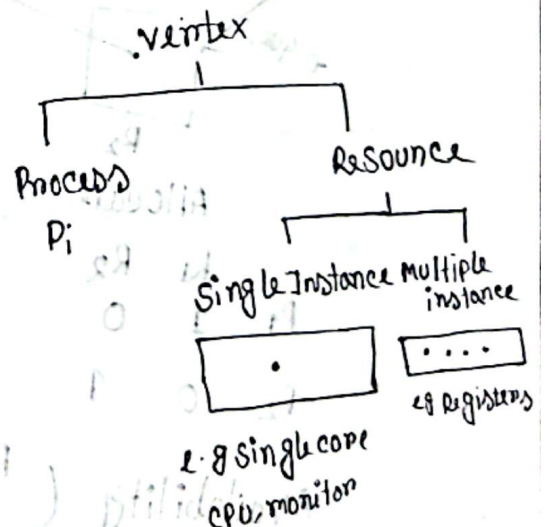
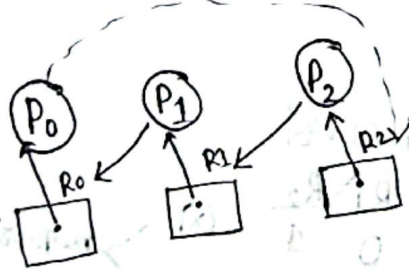
In a deadlock

4 conditions for a deadlock

- ① Mutual exclusion: only 1 process uses the resource at a time
- ② No preemption on Resource: A resource is only released when a process is done with it
- ③ Hold & wait: A process is holding one or more resources & waiting for additional resources that are being held by other processes.
- ④ Circular wait: A set of waiting processes $\{P_0, \dots, P_n\}$ such as P_0 is waiting for resource(s) held by P_1 , P_1 is waiting for resource(s) held by $P_2, \dots, P_n$ is waiting for resource(s) held by P_0

from Graph we need to explain is there have deadlock or not?

Resource Allocation Graphs (RAGs)

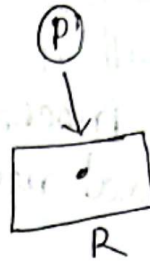


Edge

Assign

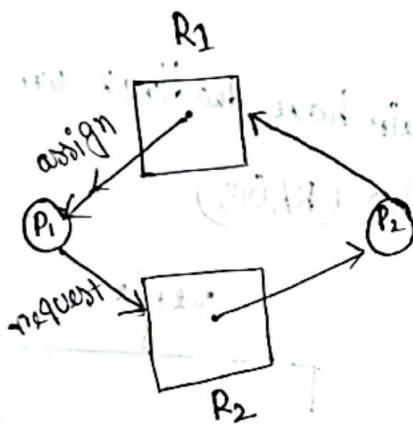
Request

1 instance taken
by P



single instance

will there be a
deadlock?

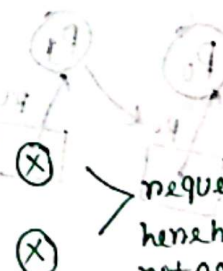


Allocate

	R ₁	R ₂
P ₁	1	0
P ₂	0	1

Request

R ₁	R ₂
0	1
1	0



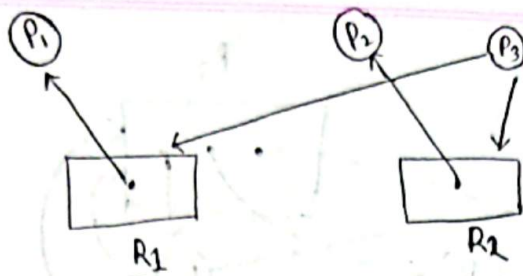
request (0 0)
here have (0, 1)
not accepted

Availability $\begin{pmatrix} R_1 & R_2 \\ 0 & 0 \end{pmatrix}$

IF RAG has a cycle & every resource is single-
instance there will always be a deadlock.

*SI & CW = Deadlock (True)
Multi Instance & CW Phase (No Deadlock)

→ Single Instance Resource



*if RAG has circular wait (cycle)
⇒ always deadlock - True
*if RAG has no cycle
then no deadlock - True

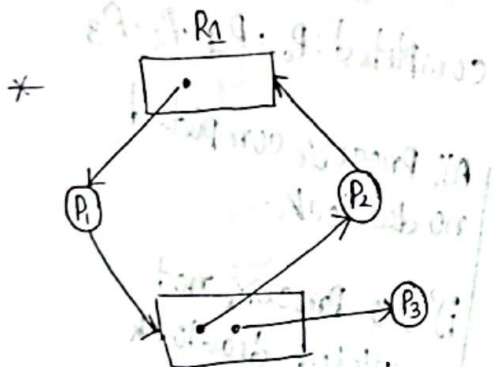
	Allocate	
	R ₁	R ₂
P ₁	1	0
P ₂	0	1
P ₃	0	0

	Request	
	R ₁	R ₂
P ₁	0	0 ✓
P ₂	0	0 ✓
P ₃	1	1 ✓

Availability	R ₁	R ₂
	0	0
+ 1	1	0
+ 0	1	1
	1	1
+ 0	1	1
	1	1

Final availability (1,1)

complete
*all Process & No deadlock



	Allocate		Request	
	P ₁	R ₂	R ₁	R ₂
P ₁	1	0	0	1
P ₂	0	1	1	0
P ₃	0	1	0	0

Availability (0,0)

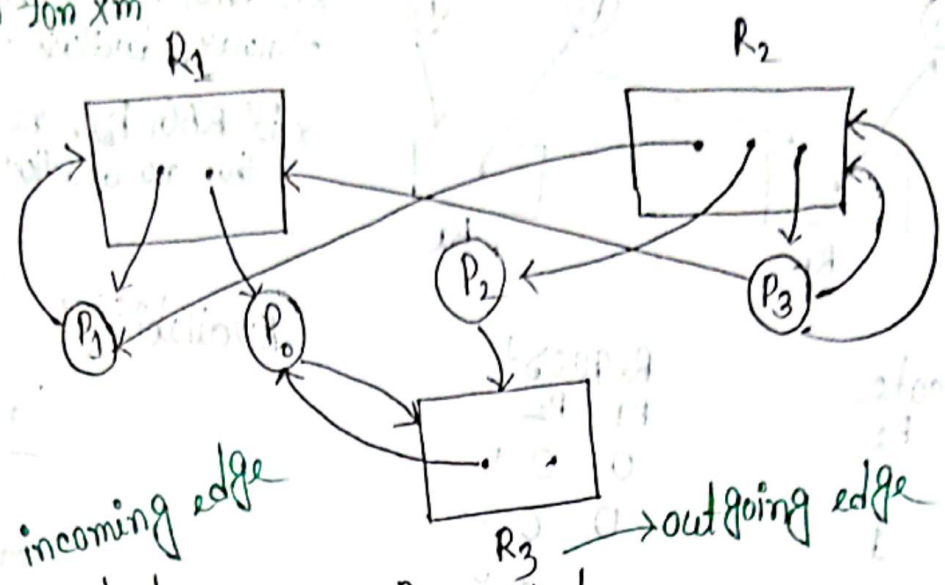
P₃, P₁, P₂

	R ₁	R ₂
	0	0
	0	1
	0	1
	1	0
	1	1

12-04-26

always checking incoming arrow

#Question for xm



	Allocated			Requested		
	R_1	R_2	R_3	R_1	R_2	R_3
P_0	1	0	1	0	0	1
P_1	1	1	0	1	0	0
P_2	0	1	0	0	0	1
P_3	0	1	0	1	2	0

check the request instance
↑ allocated on not allocated R_1, R_2, R_3

Availability 0 $(0, 0, 1)$
 $+ 1, 0, 1$

 $(1, 0, 2)$

Completed: P_0
 Availability 1 $(1, 1, 0)$

 $(2, 1, 2)$

Availability 2 $(2, 1, 2)$
 $+ 0, 1, 0$

 $(2, 2, 2)$

Availability 3 $(2, 2, 2)$
 $+ 0, 1, 0$

 $(2, 3, 2)$

Final Availability

completed: P_0, P_1, P_2, P_3

All Process completed
no deadlock.
if one Process not
completed deadlock
happend

Uplo this Quiz-3